PROJECT SYNOPSIS

Dept: Artificial Intelligence and Machine Learning

"Dynamic Context Awareness for knowledge stagnated LLMs using Self-Evolving Large Language Models(SELs) "

PROJECT TEAM

1. Joy Palit – 1IC23AI021

PROJECT GUIDE

Dr. Rama Krishna K
(Prof. and HOD, Artificial Intelligence and Machine Learning Department)

TABLE OF CONTENTS

S.No.	Chapter	Page No.
1	ABSTRACT	1
2	LITERATURE SURVEY	2-3
3	PROBLEM STATEMENT	4
4	SYSTEM REQUIREMETS 1.1 HARDWARE REQUIREMETS 1.2 SOFTWARE REQUIREMETS	5
5	PROPOSED METHODOLOGY	6-8
6	EXPECTED OUTCOMES	9

ABSTRACT

Large Language Models (LLMs) are traditionally trained in a static, offline setting using large-scale corpora, after which their parameters remain fixed during deployment. Although this paradigm enables strong generalization, it restricts the model's ability to incorporate newly emerging knowledge, adapt to domain shifts, or refine task-specific capabilities without expensive retraining or external supervision. This limitation is particularly significant for lightweight models operating in resource-constrained environments, where frequent large-scale fine-tuning is computationally impractical.

This project presents a **Self-Evolving Large Language Model (SEL)**, a compact transformer-based architecture designed to iteratively refine its internal parameters through a closed-loop self-adaptation mechanism. Unlike conventional approaches that rely on external datasets or auxiliary adaptation modules, the proposed framework enables the model to actively participate in its own improvement process. Given novel inputs or tasks, the model generates structured self-training samples, reformulated knowledge representations, and optimization directives that guide subsequent parameter updates.

These self-generated artifacts are incorporated through supervised fine-tuning, producing persistent weight updates that accumulate across adaptation cycles. A performance-driven feedback signal, derived from downstream evaluation metrics, regulates the evolution process and incentivizes the generation of high-utility training data. This feedback loop establishes a dynamic interaction between generation, evaluation, and optimization, allowing the system to progressively enhance its knowledge integration and generalization capacity.

The proposed architecture establishes a scalable and modular foundation for compact, self-improving language models capable of autonomous, persistent, and adaptive learning. This work represents a practical step toward developing intelligent systems that can evolve continuously in response to new information without requiring repeated large-scale retraining.

LITRATURE SURVEY

This chapter presents a detailed review of existing research and systems related Self-Evolving Large Language Models (SELs). It analyses current methodologies, identifies their thesis, and establishes the necessity for the proposed valuable framework.

Methodology	Reference papers	Cited Contributions	Limitations addressed by SEL
Static Training Paradigms	• Language models are Few Shot Learners • Self-Adapting Large Language Model • BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding	Trained offline with pretraining and fine-tuning; No post-deployment weight adaptation	Cannot dynamically update knowledge Retraining is costly and slow
Continual/Incremental Learning	• Overcoming Catastrophic Forgetting in neural networks • Gradient Episodic Memory for Continual Learning	Optimize model outputs using signal rewards Directly optimizes reasoning and performance Preserve prior knowledge by learning sequentially	Require Memory storage and high computation cost Focuses on output quality not optimization Target final answers not weight adaptation strategy
Self-Training/ Semi-Supervised Learning	• The Simple and Efficient Semi-Supervised Learning Method for Deep Neural Networks • Self-training with Noisy Student Improves ImageNet Classification	Use pseudo-labels to augment training data	Do not optimize hyperparameter dynamically Lack of structured reusable training directives
Synthetic Data & Instruction Tuning	• Aligning Language Model with self-generated instructions	Generate artificial data for training and fine tuning	Often heuristic-based and manually tuned

	• Training models to follow instructions with human feedback		No direct optimization for downstream utility
Knowledge Editing / Updating	• Editing Factual knowledge in Language Models • Locating and Editing Factual Associations in GPT • Mass Editing in a Transformer	Inject or modify factual knowledge via generated finetuning data	Limited optimization strategy Not a reward driven program Often static prompting formats
Test time Training	• Test Time Training with self-supervision for generalizing under Distribution shifts	Temporarily adapt weights using input specific updates	Short term adaptation only Not optimized through multi step reward feedback
Meta-Learning/ Hypernetworks	• Deep reinforcement Learning from human preferences • Constitutional Learning from human preferences	Learn how to adapt quickly across tasks Generate dynamic parameter adapters conditioned on input Optimize model outputs using reward signals	Often Uses external adapters (LoRA, hypernetworks) Limited general generative update mechanism Adds architectural complexity and not fully self-referential
Self-Improvement/ Self-Rewarding	• Hypernetworks • Low-Rank Adaptation of LLMs	Use model as reward evaluator	Limited by model's own evaluation bias Lacks structured interaction with external data Catastrophic forgetting of the model

PROBLEM STATEMENT

Small-scale transformer models lack mechanisms for autonomous self-improvement after initial training. Although they can be fine-tuned for specific tasks, adaptation remains externally driven, requiring curated datasets, manual supervision, or additional modules. These models cannot independently detect knowledge gaps, generate corrective training data, or update their own parameters through a structured feedback process.

Therefore, the fundamental research problem addressed in this work is:

How can a lightweight language model be designed to autonomously generate its own fine-tuning data and update itself through a closed-loop mechanism for continuous self-evolution without external supervision?

This project addresses the problem of designing a lightweight Large Language Model that can self-evolve through a closed-loop framework. The goal is to enable the model to autonomously generate fine-tuning data, evaluate its performance, and iteratively refine its parameters—achieving continuous adaptation and knowledge integration without relying on external supervision or retraining pipelines.

SYSTEM REQUIREMENTS

- **HARDWARE REQUIREMENTS**

Components	Minimum Requirements	Recommended
Processor	8-core CPU(Apple M-1)	24 cores(Intel i9)
RAM	8GB	64GB
GPU	MPS or RTX 2060(6GB)	RTX 4060
Storage	50-100 GB	200 GB
Training Time	24-72 hours	120+ hours

- **SOFTWARE REQUIREMENTS**

Components	
Operating System	• macOS(13.0+) • Linux(if using cloud GPU)
Framework	• Python(3.10+) • PyTorch(2.x) • Metal PS(for GPU accleration)
Tokenization and Data Processing	• Tokenizers • Dataset • Numpy • Pandas
Training Utilities	• Transformers • Accelerate • Tqdm
Evaluation Metrices	• nltk • rouge-score • scikit-learn • matplotlib • seaborn
Self-Evolution Components	• tr1 • stable-baseline3

PROPOSED METHODOLOGY

The proposed Self-Evolving Large Language Model (Mini LLM from Scratch) is designed as a closed-loop adaptive system in which generation, evaluation, and parameter optimization operate iteratively. The methodology consists of five major components: base model construction, self-edit generation, structured adaptation pipeline, feedback-driven optimization, and iterative evolution cycles.

- **Step 1-Base LLM Architecture**

The foundation of the system is a compact transformer-based autoregressive language model implemented from scratch. The architecture includes:

- Token embedding layer
- Positional encoding mechanism
- Multi-head self-attention blocks
- Feed-forward networks with layer normalization
- Output projection with softmax

The model is pretrained on a moderately sized, curated corpus to learn general linguistic structure. Pretraining uses standard next-token prediction with cross-entropy loss. The goal at this stage is not scale, but stability and interpretability, ensuring the architecture is lightweight enough for iterative fine-tuning on limited hardware.

- **Step 2-Self-Edit Generation Mechanism**

When exposed to a novel input, task, or new knowledge sample, the model generates a structured “self-edit.” This is a formatted output containing:

- Reformulated training examples
- Synthetic question–answer pairs
- Knowledge restatements
- Error-correction prompts
- Suggested learning rate or optimization hints

Instead of generating plain text, the model outputs structured adaptation artifacts in a predefined schema. This ensures compatibility with the fine-tuning pipeline.

The output is converted into supervised training samples.

- **Step 3-Structured Fine-Tuning Pipeline**

The generated self-training samples are filtered and validated through automated checks:

- Format validation

- Perplexity threshold filtering
- Consistency checks
- Redundancy removal

After validation, the filtered dataset D_{self} is used to perform supervised fine-tuning:

$$\theta_{t+1} = \theta_t - \eta \nabla L(D_{self})$$

Where:

η = learning rate

L = cross-entropy loss

Fine-tuning is performed for a limited number of steps to prevent catastrophic drift. Parameter-efficient tuning techniques (e.g., low-rank updates or partial layer updates) may be used to maintain stability.

- **Step 4-Feedback-Driven Evaluation Loop**

After each adaptation cycle, the updated model is evaluated on:

- Few-shot benchmark tasks
- Domain-specific evaluation sets
- Perplexity on held-out validation data
- Task accuracy metrics

A reward signal R is computed based on improvement relative to previous performance:

$$R = \text{Performance}(\theta_{t+1}) - \text{Performance}(\theta_t)$$

This reward signal influences future self-edit generation, either through reinforcement-style fine-tuning or weighted sampling strategies.

The objective is to maximize long-term improvement across adaptation cycles rather than single-step performance.

- **Step 5-Iterative Self-Evolution Framework**

The entire system operates as a closed-loop pipeline:

- Receive new input or knowledge
- Generate structured self-edit
- Validate and construct training dataset
- Fine-tune model parameters
- Evaluate downstream performance
- Compute feedback signal
- Repeat

Across multiple cycles, parameters evolve:

$\theta_0 \rightarrow \theta_1 \rightarrow \theta_2 \rightarrow \ldots \rightarrow \theta_n$

Persistent weight updates accumulate improvements over time.

- **Step 6-Stability and Drift Control Mechanisms**

  To prevent catastrophic forgetting or unstable divergence, the system incorporates:

  Regularization constraints

  Small learning rates

  Validation-based early stopping

  Optional replay of core pretraining samples

  These controls ensure that adaptation improves task performance without degrading foundational language capability.

- **Step 7-Experimental Setup**

  Experiments will be conducted in three stages:

- Baseline evaluation of pretrained Mini LLM
- Controlled self-evolution with incremental knowledge
- Comparative performance analysis across cycles
- Performance metrics include:
- Perplexity reduction
- Accuracy on few-shot tasks
- Knowledge recall accuracy
- Training efficiency (time and memory usage)

The proposed methodology introduces a compact transformer architecture capable of self-directed adaptation through structured self-generated training artifacts and a performance-driven feedback loop. Unlike traditional continual learning systems, this approach integrates generation and optimization into a unified, self-evolving framework without reliance on auxiliary networks or continuous human supervision.

EXPECTED OUTCOMES

The proposed Self-Evolving Large Language Model (SEL) is expected to demonstrate the feasibility of autonomous, closed-loop adaptation within a compact transformer architecture trained from scratch. The primary outcome of this work will be the successful implementation of a lightweight Mini LLM capable of generating structured self-training data and optimization directives in response to novel inputs or tasks.

- **Functional Self-Evolution Mechanism**
    - Create a functional feedback loop where the model generates its own training data.
    - Apply supervised fine-tuning on the generated artifacts.
    - Persistently update model parameters after each cycle.
    - Repeat adaptation cycles to refine performance.
    - Measure and confirm continuous task improvement to validate the self-learning framework.
- **Improved Incremental Knowledge Integration**
    - Build a feedback loop where the model generates its own training data.
    - Fine-tune the model using this data.
    - Persistently update parameters after each cycle.
    - Repeat the process to improve performance.
    - Validate effectiveness through measurable task improvements.
- **Enhanced Few-Shot Generalization**
    - Improve few-shot task adaptation through iterative refinement.
    - Track performance using metrics like accuracy, perplexity, and task-specific scores.
    - Demonstrate consistent improvement across evolution stages.
    - Confirm stronger generalization from limited examples.
- **Computational Efficiency in Resource-Constrained Settings**
    - Ensure efficient training time, memory usage, and computational overhead.
    - Maintain performance during adaptation cycles despite lightweight architecture.
    - Support deployment on modest hardware (e.g., consumer GPUs, Apple Silicon).
    - Validate suitability for compact and resource-constrained environments.
- **Reduction in External Supervision Dependence**

  The framework is expected to reduce reliance on manually curated datasets and external adaptation modules. By leveraging its intrinsic generative capability to produce training signals, the model should demonstrate partial autonomy in directing its own improvement process.